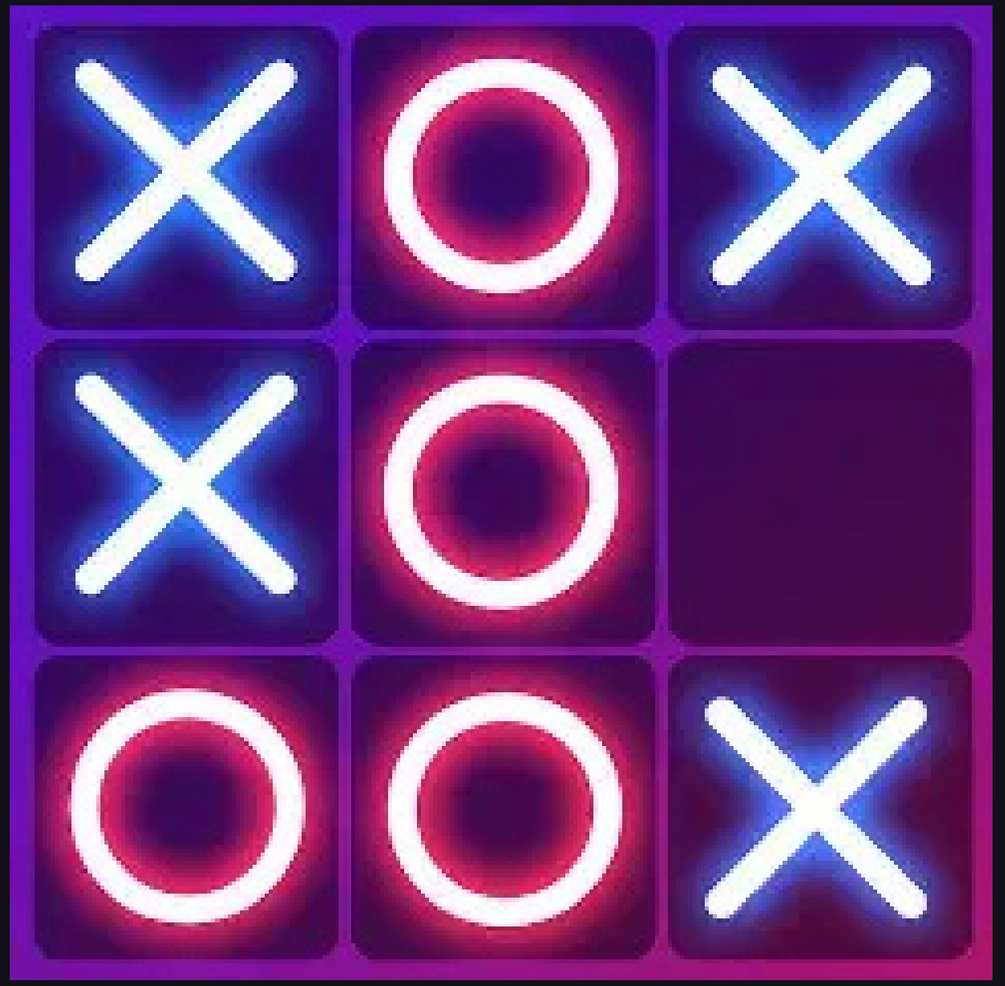
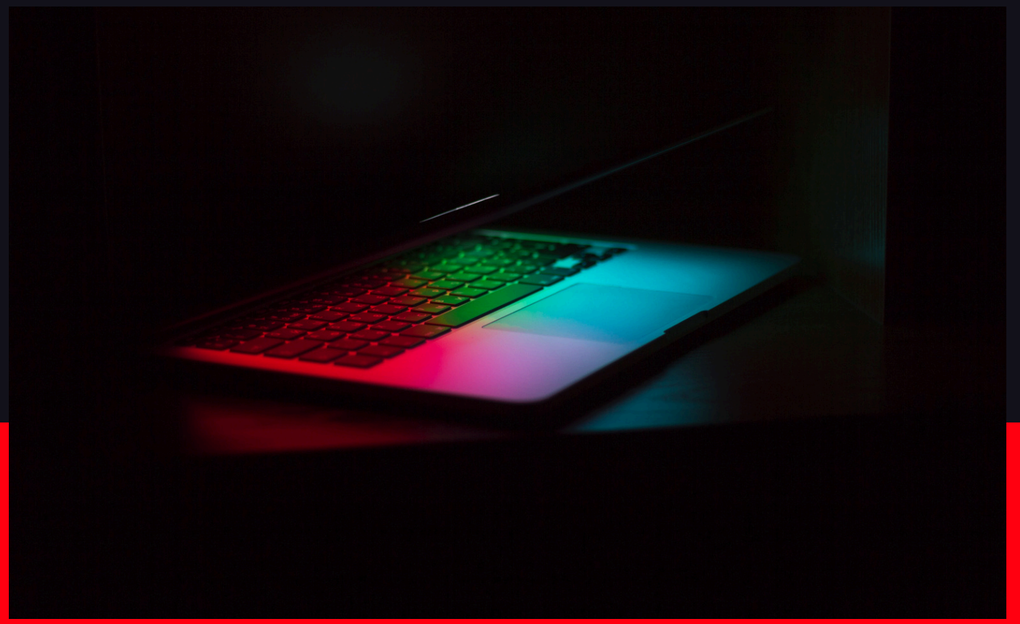
INTELLIGENT TIKTAK TOE

BY: EISHAAN KUMAR



What it does

I built a console Tic Tac Toe game in C++ where you play against a real AI running on your own computer. You are X and the AI is O. It uses a program called Ollama to run a language model locally - no internet needed. The AI reads the current board state and open spots, then decides its move from that.



Functions

I split the program into separate functions that each do exactly one job. showBoard() prints the grid, checkWin() checks all 8 win conditions, spotOpen() validates a move, and getAIMove() handles the entire Ollama side. I designed it this way so the code is easy to read and if something breaks I know exactly which function to look at.

```
void showBoard(vector<char>& board) { ... }
bool checkWin(vector<char>& board, char player) { ...
bool boardFull(vector<char>& board) { ... }
bool spotOpen(vector<char>& board, int spot) { ... }
string getOpenSpots(vector<char>& board) { ... }
string getBoardText(vector<char>& board) { ... }
int getAIMove(vector<char>& board) { ... }
```

Vectors

The board is stored as a vector of 9 chars. It starts holding the characters 1 through 9 so the player can see the spot numbers, and those get replaced with X or O as the game goes on. I used a vector instead of a regular array because vectors know their own size, so I can use board.size() instead of hardcoding 9 everywhere.

```
vector<char> board = {'1','2','3','4','5','6','7','8','9'};

for (int i = 0; i < board.size(); i++) {
    if (board[i] != 'X' && board[i] != 'O') return false;
}
```

Loops

The game runs inside a while loop that keeps going until gameOver is true. For loops inside boardFull() and getOpenSpots() check every cell on the board. There is also a for loop inside getAIMove() that scans the response field from Ollama's JSON to find the number the AI chose.

```
while (gameOver == false) {
    // main game loop
}

for (int i = 0; i < board.size(); i++) {
    if (board[i] != 'X' && board[i] != 'O') return false;
}

for (int i = numStart; i < numStart + 5; i++) {
    if (answer[i] >= '1' && answer[i] <= '9') { ... }
}
```

Conditionals

If statements control almost every decision in the game. checkWin() has 8 of them covering every row, column, and diagonal. spotOpen() checks if a spot is in range and not taken. There is also input validation using cin.fail() - if the player types a letter instead of a number it catches it and asks them to try again instead of crashing.

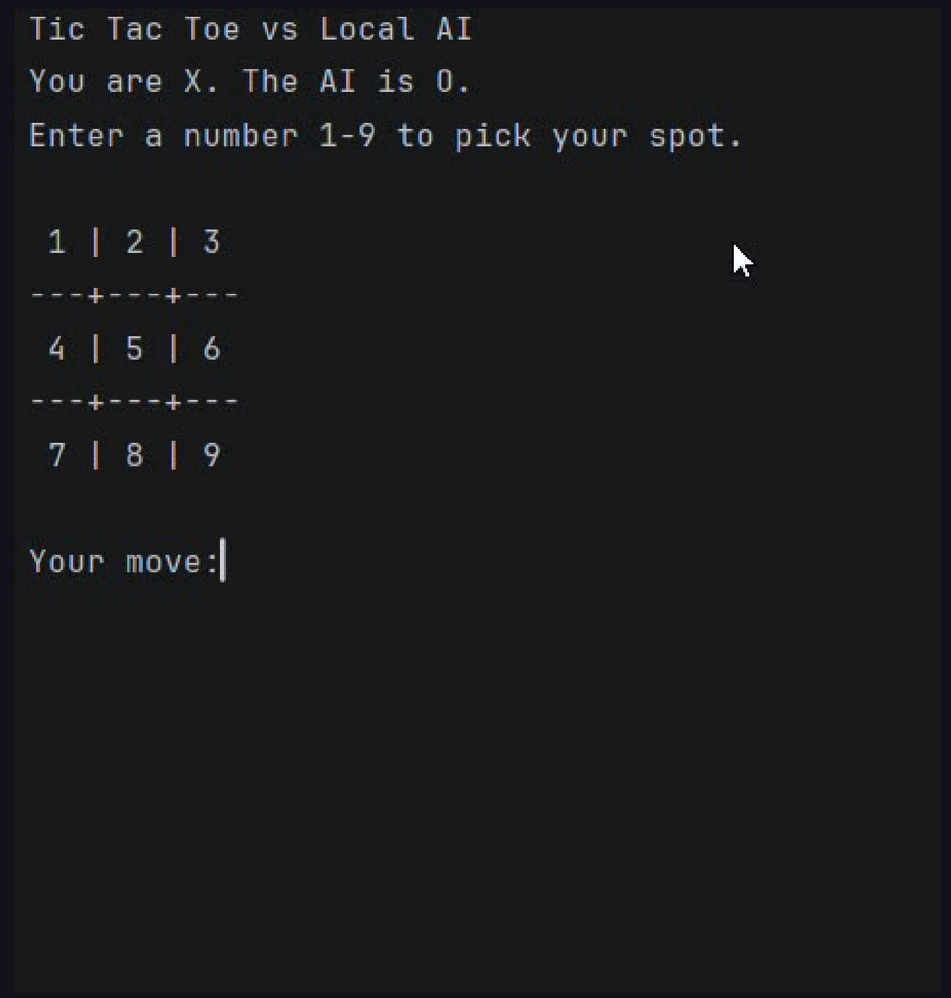
```
if (board[0]==player && board[1]==player && board[2]==player) return true;

if (cin.fail()) {
    cin.clear();
    cout << "Please enter a number." << endl;
}
```

New Research: Local AI with Ollama

My new research was connecting C++ to a locally running AI model. I used Ollama which lets you run language models on your own machine. My program builds a text prompt with the board state and open spots, sends it to Ollama using curl, gets back a JSON response, saves it to a file, then pulls the number out of the response field using string.find(). I had to learn what an API is, how HTTP requests work, and how to read JSON - none of that was covered in class.

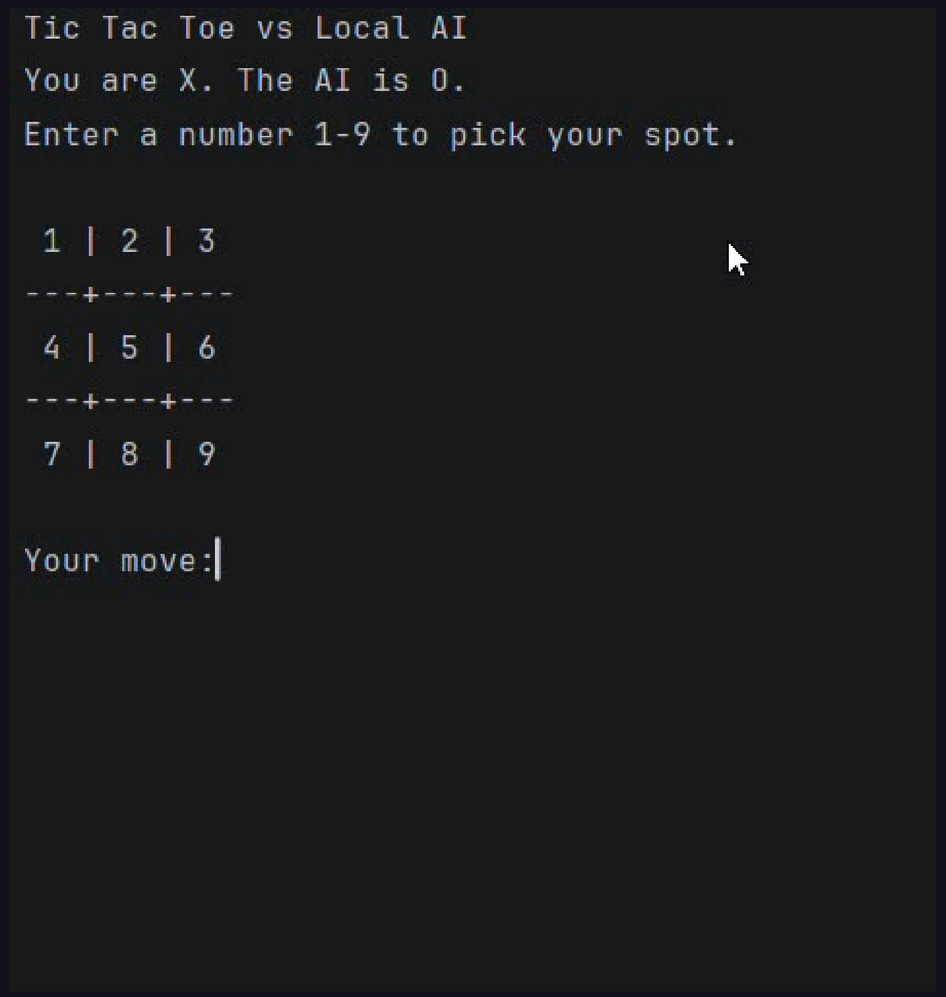
```
string command =
    "curl -s http://localhost:11434/api/generate "
    "-H \"Content-Type: application/json\" "
    "-d \"{\\\"model\\\":\\\"gemma3:4b\\\",\\\"prompt\\\":\\\"" + prompt +
    "\\\",\\\"stream\\\":false}\" > ai_response.txt";

system(command.c_str());

string responseTag = "\"response\":\"";
int tagPos = answer.find(responseTag);
```

Demo


Tic Tac Toe vs Local AI
You are X. The AI is O.
Enter a number 1-9 to pick your spot.
1 | 2 | 3
---+---+---
4 | 5 | 6
---+---+---
7 | 8 | 9
Your move:

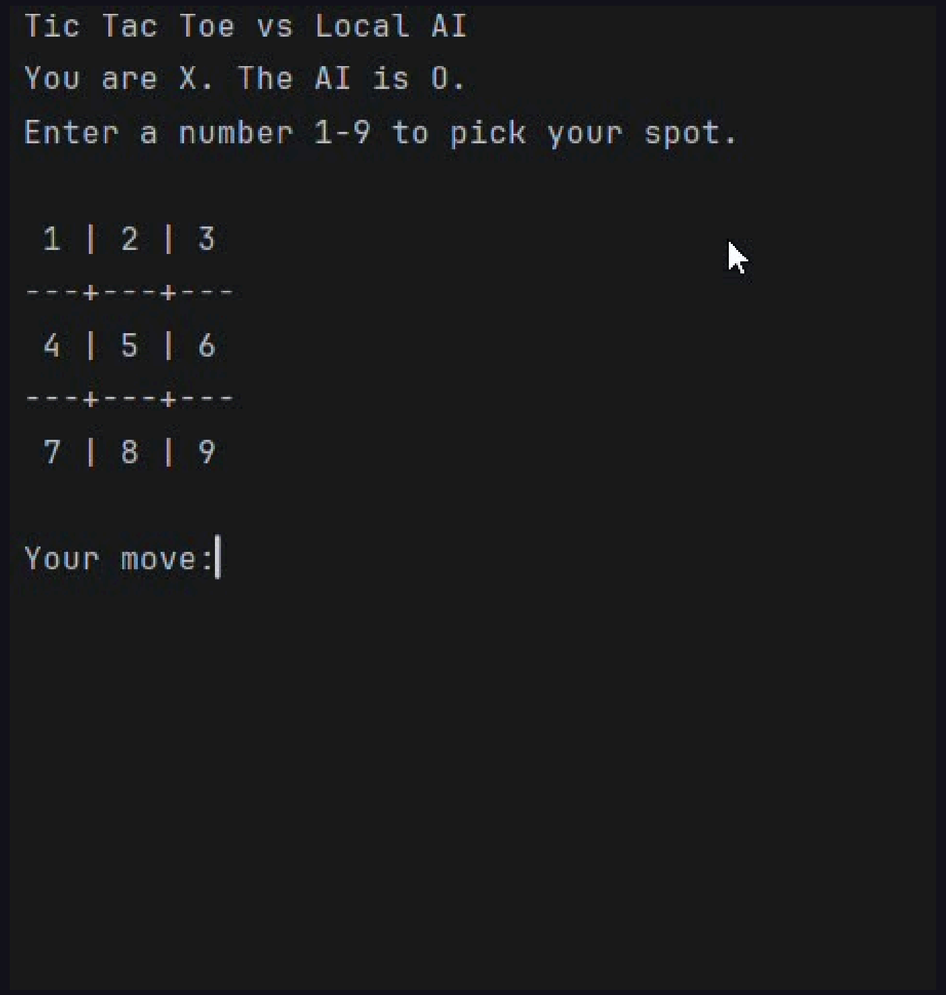
How I chalenegd myself

Hooking C++ up to a real AI was completely new to me. I had to figure out what an API is, how curl sends HTTP requests, and how to parse a JSON response without any library. I also hit a bug where my parser was reading numbers from the wrong part of the JSON and had to fix it by targeting the response field specifically with string.find().



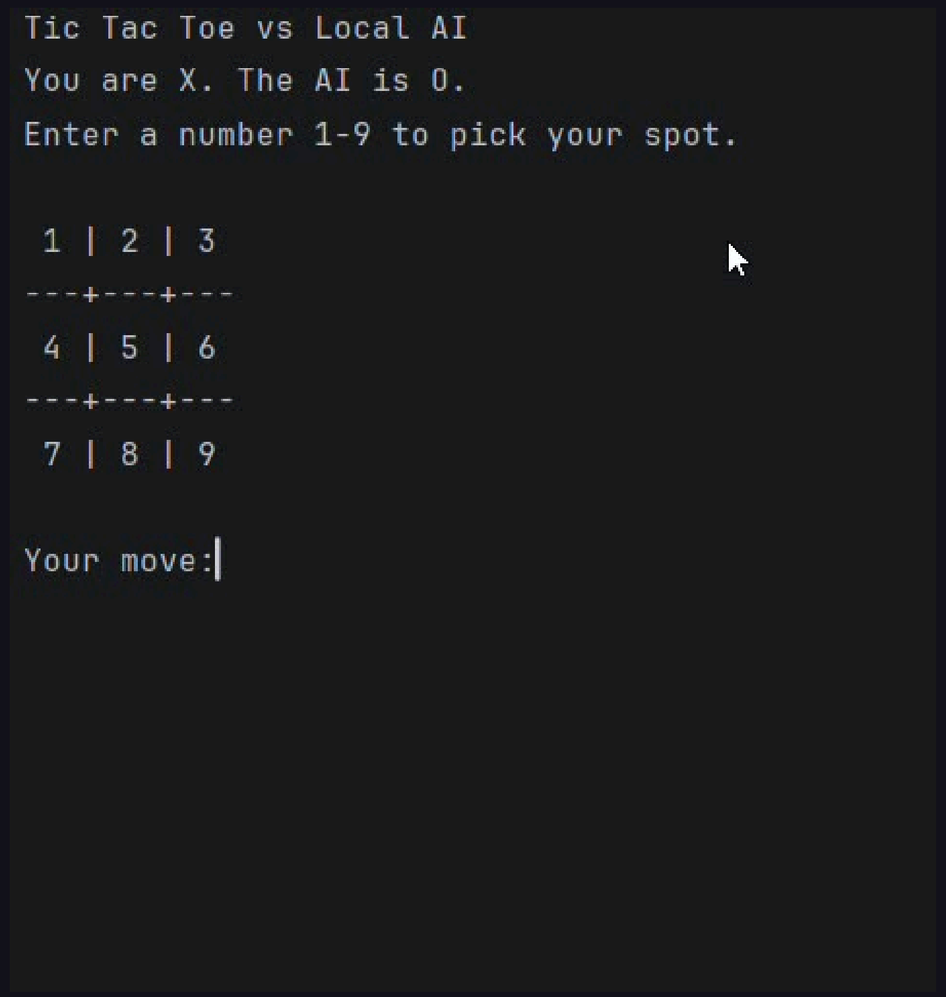
Obstacles

Setting up Ollama took trial and error. The biggest bug was that my code was finding numbers in the JSON context array instead of the actual AI response, making it always do the same moves. I fixed it using string.find() to locate the response field first. I also had to handle bad player input so typing a letter doesn't crash the game.



Next Version

Use a larger smarter model for better AI strategy. Add easy, medium, and hard difficulty. Let the player choose X or O. Add color to the board using terminal color codes. Track score across multiple rounds.



Citations

- Ollama documentation - ollama.com
- Gemma 3 model by Google
- Stack Overflow - for the curl command syntax and cin.ignore line
- cppreference.com - for vector and string methods
- YouTube - for learning how to set up and run Ollama locally
- Chatgpt for finding out why my JSON parsing wasnt working